

.NET

File and Stream I/O

1. Fundamental Concepts

- The System.IO namespace has been a core part of .NET since version 1.0 (2002), providing comprehensive APIs for file system operations and stream-based I/O.
 - .NET Framework 2.0 (2005): Added FileSystemWatcher and improved serialization
 - .NET Framework 4.0 (2010): Introduced memory-mapped files
 - .NET Core 3.0 (2019): Added high-performance System.IO.Pipelines
 - .NET 6 (2021): Improved async file operations

1.1 Core Namespaces

- **System.IO**: Basic file and directory operations
- **System.Text**: Encoding/decoding text streams
- **System.IO.Compression**: For ZIP/GZIP handling

1.2 Stream Based Architecture

- The .NET I/O system is built on a stream-based model that provides:
 - **Abstraction layer** over various storage mediums
 - **Uniform interface** for sequential byte access
 - **Buffering capabilities** for performance optimization

2. File System Operations

- The File and Directory classes provide static methods for common operations:
 - File: Create, copy, delete, move, and open files
 - Directory: Create, move, and enumerate directories
 - Path: Cross-platform path manipulation methods

2.1 File Class (Static Methods)

```
// Basic operations
File.WriteAllText("data.txt", "Hello World\n");
File.AppendAllText("data.txt", "Bye World\n");
string content = File.ReadAllText("data.txt");
bool exists = File.Exists("data.txt");

// Advanced scenarios
File.Copy("source.txt", "dest.txt", overwrite: true);
File.SetAttributes("data.txt", FileAttributes.Hidden);
```

2.2 Directory Management

```
// Directory operations
Directory.CreateDirectory("logs");
var files = Directory.EnumerateFiles("docs", "*.pdf");

// Special folders
string docsPath =
Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
```

3. Stream-Based Operations

3.1 Core Stream Classes

Class	Purpose
<code>FileStream</code>	Physical file access
<code>MemoryStream</code>	In-memory byte storage
<code>BufferedStream</code>	Performance optimization
<code>NetworkStream</code>	Network communication
<code>CryptoStream</code>	For encryption/decryption

3.2 Traditional Synchronous IO

- Traditional blocking operations suitable for:
 - Small files
 - Non-UI applications
 - Simple scripting scenarios

3.3 Write/Read Binary Files

```
// Writing to file
using (FileStream fs = File.Create("data.bin"))
using (BinaryWriter writer = new BinaryWriter(fs))
{
    writer.Write(42);
    writer.Write(3.14);
}

// Reading from file
using (FileStream fs = File.OpenRead("data.bin"))
using (BinaryReader reader = new BinaryReader(fs))
{
    int number = reader.ReadInt32();
}
```

```
double pi = reader.ReadDouble();  
}
```

3.4 Write/Read Text Files

```
// Writing text  
using (StreamWriter writer = new StreamWriter("log.txt", append: true))  
{  
    writer.WriteLine($"{DateTime.Now}: Application started");  
}  
  
// Reading lines  
using (StreamReader reader = new StreamReader("data.csv"))  
{  
    while (!reader.EndOfStream)  
    {  
        string line = reader.ReadLine();  
        // Process line  
    }  
}
```

3.5 Stream Processing Patterns

A. Basic Stream Usage

- The standard disposable pattern for streams ensures:
 - Proper resource cleanup
 - Exception safety
 - Buffer management

B. Buffering Strategies

- Critical for performance optimization:
 - Default buffer sizes (4KB typically)
 - Custom buffering for large files
 - Flush considerations for writers
- Example:

```
using (FileStream fileStream = new FileStream("output.txt", FileMode.Create))  
{  
    using (BufferedStream bufferedStream = new BufferedStream(fileStream)) {  
        using (StreamWriter streamWriter = new StreamWriter(bufferedStream))  
        {  
            streamWriter.WriteLine("First line of text.");  
            streamWriter.WriteLine("Second line of text.");  
        }  
    }  
}
```

```
}  
}
```

C. Position and Seeking

- Random access capabilities:
 - Position property tracking
 - Seek() for arbitrary access
 - Length monitoring

4. Serialization and Formatters

- Serialization is converting each given object into sequence of bytes.
- This sequence of bytes can be written into any stream e.g. FileStream, NetworkStream, etc.
- Deserialization is converting sequence of bytes back to the object.

4.1 Binary Serialization

- `[Serializable]` attribute enables:
 - Object graph preservation
 - Type fidelity
 - Compact binary representation
- `[NonSerialized]` attribute on field:
 - makes that field non serialized (like `transient` in Java).
- **Note:**
 - BinaryFormatter is deprecated as Deserialization is insecure (may cause attack).
 - Removed in .NET 9. Needs to be enabled explicitly in .NET 8 project settings (still with a warning).
 - `<EnableUnsafeBinaryFormatterSerialization>true</EnableUnsafeBinaryFormatterSerialization>`
- Example:

```
[Serializable]  
public class Person { /* ... */ }  
  
// Serialize  
BinaryFormatter formatter = new BinaryFormatter();  
using (FileStream fs = File.Create("person.bin"))  
{  
    formatter.Serialize(fs, new Person());  
}  
  
// Deserialize  
using (FileStream fs = File.OpenRead("person.bin"))
```

```
{  
    Person p = (Person)formatter.Deserialize(fs);  
}
```

4.2 XML Serialization

- XmlSerializer provides:
 - Human-readable output
 - Schema generation
 - Interoperability
- Example:

```
XmlSerializer serializer = new XmlSerializer(typeof(Person));  
  
// Serialize to file  
using (TextWriter writer = new StreamWriter("person.xml"))  
{  
    serializer.Serialize(writer, new Person());  
}  
  
// Deserialize from file  
using (TextReader reader = new StreamReader("person.xml"))  
{  
    Person p = (Person)serializer.Deserialize(reader);  
}
```

4.3 JSON Serialization (System.Text.Json)

- Modern default choice with:
 - System.Text.Json (high performance)
 - Newtonsoft.Json (rich features)
 - Async streaming support
- Example:

```
// Serialize  
string json = JsonSerializer.Serialize(new Person());  
File.WriteAllText("person.json", json);  
  
// Deserialize  
string jsonText = File.ReadAllText("person.json");  
Person p = JsonSerializer.Deserialize<Person>(jsonText);
```

5. Advanced Scenarios

5.1 Asynchronous I/O

- Modern non-blocking pattern offering:
 - Better scalability
 - UI responsiveness
 - Efficient resource utilization
- Example:

```
async Task ProcessFileAsync() {  
    using (StreamReader reader = new StreamReader("largefile.txt")) {  
        string content = await reader.ReadToEndAsync();  
        // Process content  
    }  
}
```

5.2 File System Watcher

- FileSystemWatcher provides events for:
 - File creations/modifications
 - Directory changes
 - Rename operations
- Example:

```
FileSystemWatcher watcher = new FileSystemWatcher("C:\\WatchFolder");  
watcher.Created += (s, e) => Console.WriteLine($"Created: {e.Name}");  
watcher.EnableRaisingEvents = true;
```

6.3 Memory-Mapped Files

- For extremely large files:
 - Efficient random access
 - Shared memory between processes
 - Native OS integration
- Example:

```
using (var mmf = MemoryMappedFile.CreateFromFile("large.bin"))  
using (var accessor = mmf.CreateViewAccessor()) {  
    int value = accessor.ReadInt32(position: 0);  
}
```

6. Best Practices

6.1 Resource Management

1. **Always dispose streams** (use `using` blocks)
2. **Flush writers** when needed (or use auto-flush)
3. **Handle exceptions** (`FileNotFoundException`, `UnauthorizedAccessException`)

6.2 Performance Considerations

1. **Buffer sizes**: Optimal defaults exist (typically 4KB)
2. **Async vs Sync**: Use async for UI apps and services
3. **File sharing**: Consider `FileShare` modes for concurrent access

6.3 Security

1. **Validate file paths** (avoid path traversal attacks)
2. **Handle sensitive data** carefully in memory
3. **Set proper permissions** when creating files

MSDN References

- [File and Stream I/O](#)
- [System.Text.Json](#)
- [Asynchronous File Access](#)

Exception Handling

1. Introduction

- Exception handling has been a fundamental part of C#, providing a structured way to handle runtime errors.
- The .NET exception model is based on a hierarchy of exception classes, all deriving from `System.Exception`.

2. Core Exception Handling Constructs

2.1 try-catch Blocks

- The fundamental structure for catching exceptions:

```
try {  
    // Code that might throw exceptions  
    File.ReadAllText("nonexistent.txt");  
}  
catch (FileNotFoundException ex) {  
    Console.WriteLine($"File not found: {ex.FileName}");  
}  
catch (Exception ex) {
```

```

        Console.WriteLine($"General error: {ex.Message}");
    }

```

2.2 finally Block

- Guarantees execution for cleanup:

```

FileStream file = null;
try {
    file = File.Open("data.txt", FileMode.Open);
    // Process file
}
finally {
    file?.Dispose(); // Always executes
}

```

2.3 using Statement

- Simplified resource cleanup (implements IDisposable):

```

using (var file = File.Open("data.txt", FileMode.Open)) {
    // Automatically disposed when block exits
}

```

3. Exception Types Hierarchy

3.1 Common Exception Types

```

// System Exceptions
catch (ArgumentNullException ex) { ... }
catch (ArgumentException ex) { ... }
catch (InvalidOperationException ex) { ... }

// IO Exceptions
catch (FileNotFoundException ex) { ... }
catch (DirectoryNotFoundException ex) { ... }

// Custom Exceptions
catch (MyCustomException ex) { ... }

```

```

System.Object
├── System.Exception
│   └── System.SystemException

```

```

├── System.NullReferenceException
├── System.ArithmeticException
│   └── System.DivideByZeroException
├── System.ArgumentException
│   ├── System.ArgumentNullException
│   └── System.ArgumentOutOfRangeException
├── System.InvalidOperationException
├── System.IndexOutOfRangeException
├── System.IO.IOException
│   └── System.IO.FileNotFoundException
├── System.Threading.ThreadAbortException
└── System.ApplicationException
    └── MyCustomException

```

3.2 Creating Custom Exceptions

```

public class InventoryException : ApplicationException {
    public int ItemId { get; }
    public InventoryException(int itemId, string message) : base(message) {
        ItemId = itemId;
    }
}

```

```

public class InventoryManagement {
    // fields
    public InventoryItem GetItemDetails(int itemId) {
        // ...
        if(notFound)
            throw new InventoryException(itemId, "Item is not in stock");
        // ...
    }
}

```

```

// Usage
try {
    InventoryManagement im = new InventoryManagement();
    // ...
    InventoryItem item = im.GetItemDetails(id);
    // ...
}
catch(InventoryException e) {
    Console.WriteLine(e.ToString());
}

```

4. Advanced Exception Handling

4.1 Exception Filters (C# 6)

- Conditional catch blocks:

```
try { ... }
catch (HttpRequestException ex) when (ex.StatusCode == 404) {
    Console.WriteLine("Resource not found");
}
```

4.2 AggregateException

- Handling multiple exceptions:

```
try {
    Parallel.ForEach(items, ProcessItem);
}
catch (AggregateException ae) {
    foreach (var ex in ae.InnerExceptions) {
        Console.WriteLine(ex.Message);
    }
}
```

5. Best Practices

5.1 Do's

```
// Catch specific exceptions
catch (SqlException ex) { ... }

// Provide meaningful messages
throw new InvalidOperationException("Connection pool exhausted");

// Log complete exception details
logger.LogError(ex, "Processing failed for {ItemId}", itemId);
```

5.2 Don'ts

```
// Avoid empty catch blocks
catch (Exception) { }

// Don't catch without action
catch (Exception ex) {
    throw; // Just rethrowing is usually pointless
}
```

```
// Avoid exception-based flow control -- This is not good practice
try {
    using (StreamReader reader = new StreamReader("your_file.txt")) {
        while (true) {
            string line = reader.ReadLine();
            if (line == null) // Alternative check for end of stream
                break;
            // Process the line
        }
    }
}
catch (EndOfStreamException ex) {
    // Handle the end of stream condition
    Console.WriteLine("End of stream reached: " + ex.Message);
}
```

6. Performance: Exception Costs

- Stack trace generation is expensive
- Avoid exceptions in normal flow
- Use Try-pattern for expected cases:

```
if (int.TryParse(input, out var number)) {
    // Success case
}
else {
    // Handle invalid input
}
```

MSDN References

- [Exception Handling](#)
- [Best Practices](#)
- [Creating Custom Exceptions](#)

Resource Management & Garbage Collection

1. Introduction

- The .NET runtime provides automatic memory management through its garbage collector (GC). The CLR memory management system handles:
 - Allocation of objects on the managed heap
 - Reclamation of unused memory
 - Compaction of surviving objects

2. Garbage Collection Fundamentals

2.1 Generational Collection

- The heap is divided into generations:
 - **Generation 0**: Short-lived objects (most collections occur here)
 - **Generation 1**: Buffer between short-lived and long-lived objects
 - **Generation 2**: Long-lived objects
 - **Large Object Heap (LOH)**: Objects > 85KB (collected less frequently)

2.2 Collection Triggers

- GC runs when:
 1. Generation 0 reaches its budget/threshold
 2. System memory is low
 3. AppDomain is unloading
 4. Explicitly called via `GC.Collect()`

2.3 Garbage Collection Process

1. Marking Phase: The GC identifies live objects by traversing the object graph starting from roots (like global variables and stack references). It marks reachable objects as "live" and identifies unreachable objects as candidates for collection.
2. Relocating Phase (optional): If the GC determines there's significant fragmentation in the heap, it updates references of live objects to new compacted locations before the compacting phase.
3. Compacting Phase: The GC reclaims the memory occupied by dead objects and compacts the remaining live objects into a contiguous block of memory. This reduces fragmentation and improves memory locality.

2.4 GC Flavors

- **Workstation GC**: Optimized for UI apps (lower latency)
- **Server GC**: Optimized for throughput (multiple CPU cores)

3. Deterministic Resource Cleanup

3.1 IDisposable Interface

- It's an interface in the System namespace.

```
public interface IDisposable {  
    void Dispose();  
}
```

- It contains a single method: `Dispose()`.
- Implementing `IDisposable` signals that a class holds resources that need explicit cleanup.

3.2 Standard Disposable Pattern

```
public class Resource : IDisposable {
    private bool _disposed = false;
    public void Dispose()
    {
        // Dispose managed resources
        _disposed = true;
        GC.SuppressFinalize(this);
    }
    ~Resource() {
        if(!_disposed)
            Dispose();
    }
}
```

3.3 Usage Patterns

```
// Explicit disposal
var resource = new Resource();
try { /* use resource */ }
finally { resource.Dispose(); }
```

```
// Using statement (preferred)
using (var resource = new Resource()) {
    // Automatic disposal
}
```

4. Finalization (Destructors)

4.1 Finalizer Syntax

```
public class Resource {
    ~Resource() { // Finalizer
        // Cleanup unmanaged resources
    }
}
```

4.2 Finalization Process

1. Object becomes unreachable
2. GC queues object for finalization
3. Finalizer thread calls finalizer
4. Memory reclaimed in next GC cycle

4.3 Performance Impact

- Finalizable objects survive first collection
- Require additional GC work
- Add latency to cleanup process

5. Best Practices

5.1 Resource Management

1. Implement `IDisposable` for types holding:
 - File handles
 - Database connections
 - Native resources
2. Prefer `using` over manual `try-finally`
3. Avoid finalizers unless absolutely necessary

5.2 GC Optimization

1. Minimize large object allocations
2. Consider object pooling for frequent allocations
3. Use `structs` for small, short-lived data
4. Avoid unnecessary references

5.3 Anti-Patterns

1. Calling `GC.Collect()` explicitly
2. Empty finalizers
3. Resurrecting (reuse/reassign) objects in finalizers
4. Overusing object pinning (`fixed` keyword)

MSDN References

- [Garbage Collection](#)
- [IDisposable Pattern](#)
- [Memory Management](#)

Indexers and Iterators

1. Indexers

Concept

- Indexers allow objects to be indexed like arrays, providing a way to access elements using the `[]` notation. They are essentially properties named "this" that take parameters.

Example

```
// consider a user defined stack of strings
public class DemoStack {
    // ...
    public string this[int index] {
        get {
            if (index < 0 || index > _top)
                throw new IndexOutOfRangeException();
            return _items[index];
        }
        set {
            if (index < 0 || index > _top)
                throw new IndexOutOfRangeException();
            _items[index] = value;
        }
    }
}
```

Usage

```
var stack = new DemoStack<string>(5);
stack.Push("First");
stack.Push("Second");
Console.WriteLine(stack[0]); // Accesses "First"
stack[1] = "Updated";        // Modifies "Second"
```

2. Iterators

Concept

- Iterators provide a way to traverse collections using `foreach` by implementing `IEnumerable` or `IEnumerable<T>`. The `yield` keyword simplifies iterator implementation.

Implementation Example

```
public class DemoStack : IEnumerable {
    // ...
    public IEnumerator GetEnumerator() {
        for (int i = _top; i >= 0; i--) {
            yield return _items[i]; // Lazy evaluation
        }
    }
}
```

Usage

```
foreach (var item in stack) {  
    Console.WriteLine(item); // Prints from top to bottom  
}
```

3. Key Features

Indexer Characteristics

1. Can be overloaded with different parameter types
2. Can have multiple dimensions (`[x,y]`)
3. Can be defined in interfaces
4. Support get-only or set-only accessors

Iterator Characteristics

1. `yield return` provides lazy evaluation
2. Maintains state between iterations
3. Can implement complex traversal logic
4. Compiler generates state machine

4. Best Practices

1. Indexers:

- Validate index parameters
- Consider performance for large collections
- Document expected behavior

2. Iterators:

- Prefer `IEnumerable<T>` over `IEnumerable`
- Avoid modifying collections during iteration
- Consider thread safety

MSDN References

- [Indexers \(C#\)](#)
- [Iterators \(C#\)](#)
- [yield \(C# Reference\)](#)